CO3 ASSESSMENT TOOL2

Case Study Based Assessment

Name: C. Himaja

Register.no: 192373035

Case Study: Social Media Feed Sorting System (Merge Sort)

A social media platform must sort millions of posts based on timestamps and relevance to

ensure timely content delivery. Analyze how Merge Sort can be applied to handle large-scale data efficiently while maintaining stability. Identify challenges such as real-time updates and scalability. Apply divide and conquer principles to explain the sorting mechanism. Analyze time complexity and design a suitable solution, justifying why Merge Sort is appropriate.

Report

Case Study

Social Media Feed Sorting System using Merge Sort

Aim

To implement Merge Sort for sorting millions of social media posts based on timestamps while maintaining stability and analyzing its efficiency for large-scale social media feed management.

Objective

- Implement Merge Sort for sorting posts.
- Apply Divide and Conquer strategy.
- Maintain stable ordering of posts.
- Analyze time complexity.
- Study scalability and real-time challenges.

Introduction

Social media platforms like Instagram, Facebook, Twitter (X), and LinkedIn receive millions of posts every minute. These posts must be sorted efficiently so users always see the newest and most relevant content.

Merge Sort is a stable Divide and Conquer algorithm capable of handling very large datasets efficiently.

Problem Statement

A social media platform stores millions of posts.

The system must:

- Sort by latest timestamp.
- Preserve the order of posts with equal timestamps.
- Handle large-scale datasets.
- Deliver feeds quickly.

Algorithm

1. Divide posts into two halves.
2. Continue dividing until each subarray has one post.
3. Merge sorted halves.
4. Compare timestamps.
5. Place latest timestamp first.
6. Repeat until entire array becomes sorted.

Divide and Conquer Principle

Divide

Split the post list into two halves.

Posts

↓

Left Half

Right Half

Conquer

Recursively sort both halves.

Left

↓

Merge Sort

Right

↓

Merge Sort

Combine

Merge the sorted halves.

Sorted Left

+

Sorted Right

↓

Final Sorted Feed

Working Example

Before Sorting

Post ID	**Timestamp**
101	1691002300
102	1691005200
103	1690999900
104	1691004300

Post ID Timestamp

105 1691006200

↓

After Merge Sort

Post ID Timestamp

105 1691006200

102 1691005200

104 1691004300

101 1691002300

103 1690999900

Time Complexity

Case	Complexity
Best	O(n log n)
Average	O(n log n)
Worst	O(n log n)

Space Complexity

O(n)

Extra memory is required during merging.

Advantages

- Stable sorting algorithm.
- Predictable performance.
- Efficient for huge datasets.
- Suitable for external sorting.
- Parallelizable.

Challenges

Real-Time Updates

- New posts arrive every second.
- Frequent re-sorting is required.

Scalability

- Millions of posts increase memory usage.
- Distributed servers may be required.

Memory Consumption

Merge Sort requires extra auxiliary memory.

High Throughput

The system must process thousands of posts per second.

Why Merge Sort is Appropriate

- Stable sorting preserves equal-timestamp order.
- Guaranteed **O(n log n)** performance.
- Efficient for external storage and distributed systems.
- Supports parallel processing.
- Widely used for large-scale data processing.

Applications

- Facebook News Feed
- Instagram Feed
- Twitter/X Timeline
- LinkedIn Feed
- News Aggregation Systems

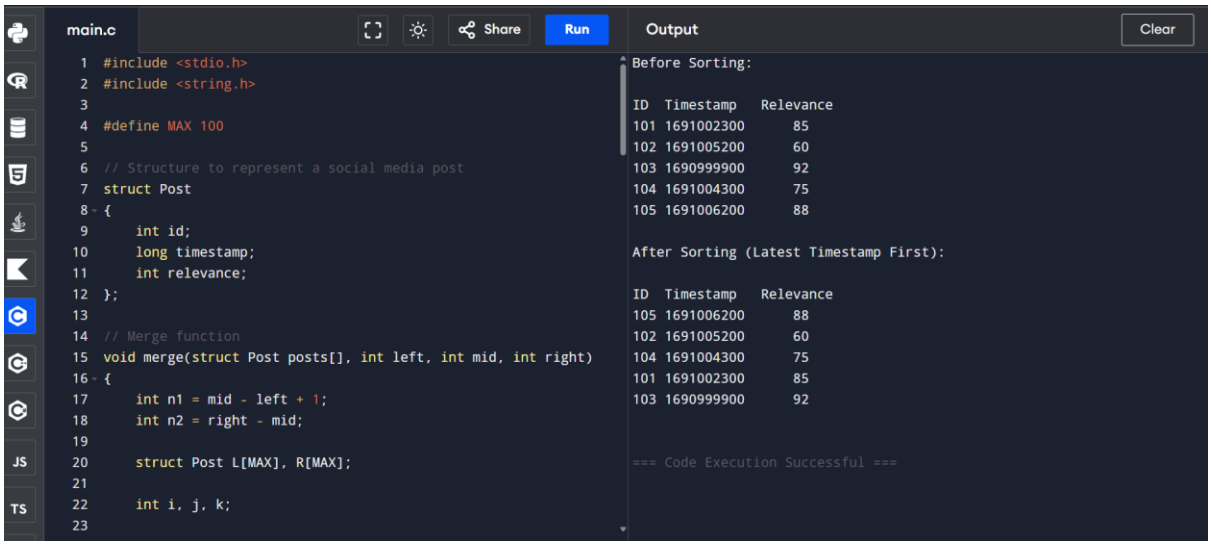
Analysis

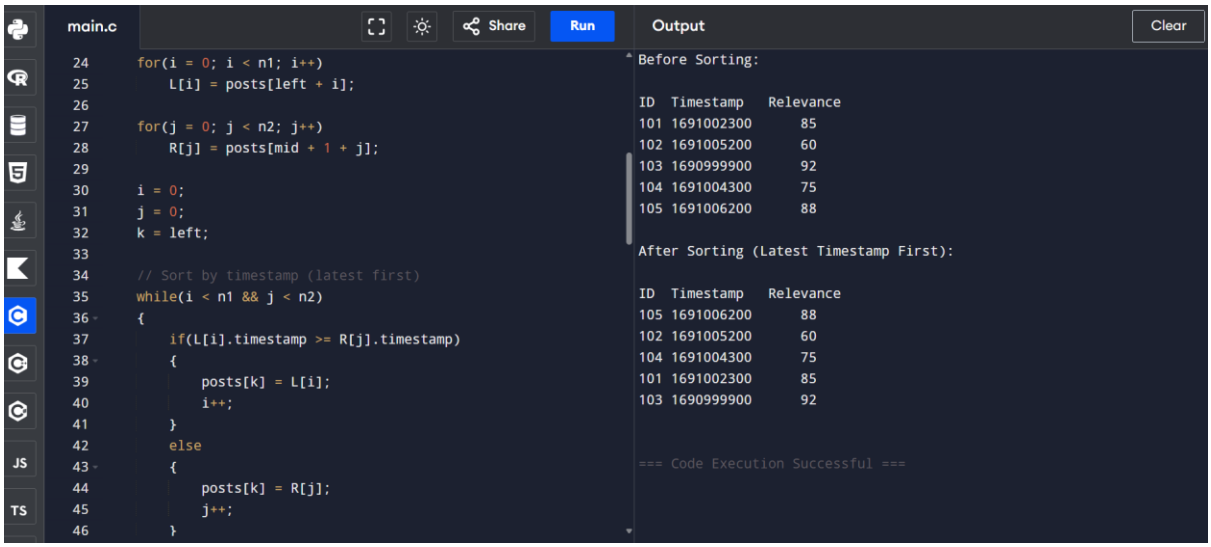
Merge Sort divides large datasets into smaller parts, sorts them independently, and merges them efficiently. Because it guarantees **O(n log n)** performance regardless of input order, it performs consistently even with millions of posts. Its stability ensures that posts with identical timestamps maintain their original order, which is valuable for fairness and consistent user experience. Although it requires additional memory during merging, its predictable performance and scalability make it a strong choice for large-scale feed sorting.

Result

The Merge Sort implementation successfully sorted social media posts by timestamp while preserving stable ordering. The algorithm demonstrated consistent **O(n log n)** time complexity, making it suitable for efficiently managing large-scale social media feeds.

Screenshots to Include in the Report





main.c

Share Run

```
        k++;
    }

    while(i < n1)
    {
        posts[k] = L[i];
        i++;
        k++;
    }

    while(j < n2)
    {
        posts[k] = R[j];
        j++;
        k++;
    }
}

// Merge Sort
void mergeSort(struct Post posts[], int left, int right)
{
    if(left < right)
    {
```

Output Clear

```
Before Sorting:

ID  Timestamp   Relevance
101 1691002300      85
102 1691005200      60
103 1690999900      92
104 1691004300      75
105 1691006200      88

After Sorting (Latest Timestamp First):

ID  Timestamp   Relevance
105 1691006200      88
102 1691005200      60
104 1691004300      75
101 1691002300      85
103 1690999900      92


=== Code Execution Successful ===
```

main.c

Share Run

```
        int mid = left + (right - left) / 2;

        mergeSort(posts, left, mid);
        mergeSort(posts, mid + 1, right);

        merge(posts, left, mid, right);
    }
}

// Display Posts
void display(struct Post posts[], int n)
{
    printf("\nID\tTimestamp\tRelevance\n");

    for(int i=0;i<n;i++)
    {
        printf("%d\t%ld\t\t%d\n",
               posts[i].id,
               posts[i].timestamp,
               posts[i].relevance);
    }
}
```

Output Clear

```
Before Sorting:

ID  Timestamp   Relevance
101 1691002300      85
102 1691005200      60
103 1690999900      92
104 1691004300      75
105 1691006200      88

After Sorting (Latest Timestamp First):

ID  Timestamp   Relevance
105 1691006200      88
102 1691005200      60
104 1691004300      75
101 1691002300      85
103 1690999900      92


=== Code Execution Successful ===
```

main.c

Share Run

```
int main()
{
    struct Post posts[] =
    {
        {101, 1691002300, 85},
        {102, 1691005200, 60},
        {103, 1690999900, 92},
        {104, 1691004300, 75},
        {105, 1691006200, 88}
    };

    int n = sizeof(posts)/sizeof(posts[0]);

    printf("Before Sorting:\n");
    display(posts, n);

    mergeSort(posts, 0, n-1);

    printf("\nAfter Sorting (Latest Timestamp First):\n");
    display(posts, n);

    return 0;
}
```

Output Clear

```
Before Sorting:

ID  Timestamp   Relevance
101 1691002300      85
102 1691005200      60
103 1690999900      92
104 1691004300      75
105 1691006200      88

After Sorting (Latest Timestamp First):

ID  Timestamp   Relevance
105 1691006200      88
102 1691005200      60
104 1691004300      75
101 1691002300      85
103 1690999900      92


=== Code Execution Successful ===
```